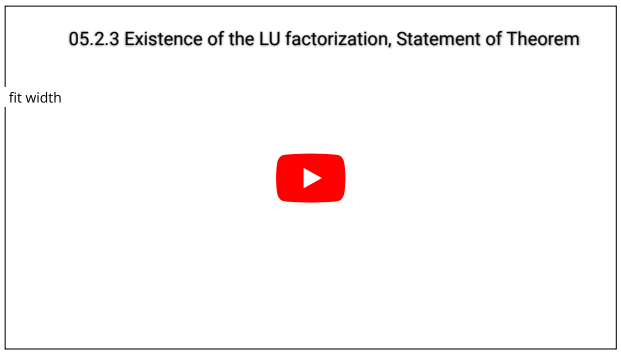
5.2.3 Existence of the LU factorization ¶



Now that we have an algorithm for computing the LU factorization, it is time to talk about when this LU factorization exists (in other words: when we can guarantee that the algorithm completes).

We would like to talk about the existence of the LU factorization for the more general case where A is an $m \times n$ matrix, with $m \geq n$. What does this mean?

Definition 5.2.3.1. Given a matrix $A \in \mathbb{C}^{m \times n}$ with $m \geq n$, its LU factorization is given by $A = LU$ where $L \in \mathbb{C}^{m \times n}$ is unit lower trapezoidal and $U \in \mathbb{C}^{n \times n}$ is upper triangular with nonzeroes on its diagonal.

The first question we will ask is when the LU factorization exists. For this, we need another definition.

Definition 5.2.3.2. Principal leading submatrix. For $k \leq n$, the $k \times k$ principal leading submatrix of a matrix A is defined to be the square matrix $A_{TL} \in \mathbb{C}^{k \times k}$ such that $A = \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right)$.

This definition allows us to state necessary and sufficient conditions for when a matrix with n linearly independent columns has an LU factorization:

Lemma 5.2.3.3. *Let $L \in \mathbb{C}^{n \times n}$ be a unit lower triangular matrix and $U \in \mathbb{C}^{n \times n}$ be an upper triangular matrix. Then $A = LU$ is nonsingular if and only if U has no zeroes on its diagonal.*

Homework 5.2.3.1. Prove Lemma 5.2.3.3.
▾ Hint ▾ Solution

The proof hinges on the fact that a triangular matrix is nonsingular if and only if it doesn't have any zeroes on its diagonal. Hence we can instead prove that $A = LU$ is nonsingular if and only if U is nonsingular (since L is unit lower triangular and hence has no zeroes on its diagonal).

- ($\Rightarrow$): Assume $A = LU$ is nonsingular. Since L is nonsingular, $U = L^{-1}A$. We can show that U is nonsingular in a number of ways:
  - We can explicitly give its inverse:
    $$U(A^{-1}L) = L^{-1}AA^{-1}L = I.$$
    Hence U has an inverse and is thus nonsingular.
  - Alternatively, we can reason that the product of two nonsingular matrices, namely L^{-1} and A, is nonsingular.
- ($\Leftarrow$): Assume $A = LU$ and U has no zeroes on its diagonal. We then know that both L^{-1} and U^{-1} exist. Again, we can either explicitly verify a known inverse of A:
  $$A(U^{-1}L^{-1}) = LUU^{-1}L^{-1} = I$$
  or we can recall that the product of two nonsingular matrices, namely U^{-1} and L^{-1}, is nonsingular.

You may use the fact that a triangular matrix has an inverse if and only if it has no zeroes on its diagonal.

Theorem 5.2.3.4. Existence of the LU factorization. *Let $A \in \mathbb{C}^{m \times n}$ and $m \geq n$ have linearly independent columns. Then A has a (unique) LU factorization if and only if all its principal leading submatrices are nonsingular.*

05.2.3 Existence of the LU factorization, Proof

fit width

Proof.

The formulas in the inductive step of the proof of Theorem 5.2.3.4 suggest an alternative algorithm for computing the LU factorization of a $m \times n$ matrix A with $m \geq n$, given in Figure 5.2.3.5. This algorithm is often referred to as the (unblocked) left-looking algorithm.

$A :=$ LU-left-looking(A)

$A \rightarrow \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right)$

A_{TL} is 0×0

while $n(A_{TL}) < n(A)$

$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right) \rightarrow \left(\begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array}\right)$

Solve $L_{00}u_{01} = a_{01}$ overwriting a_{01} with u_{01}

$\alpha_{11} := v_{11} = \alpha_{11} - a_{10}^T a_{01}$

$a_{21} := a_{21} - A_{20}a_{01}$

$a_{21} := l_{21} = a_{21}/\alpha_{11}$

$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right) \leftarrow \left(\begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array}\right)$

endwhile

Figure 5.2.3.5. Left-looking LU factorization algorithm. L_{00} is the unit lower triangular matrix stored in the strictly lower triangular part of A_{00} (with the diagonal implicitly stored).

Homework 5.2.3.2. Show that if the left-looking algorithm in Figure 5.2.3.5 is applied to an $m \times n$ matrix, with $m \geq n$, the cost is approximately $mn^2 - \frac{1}{3}n^3$ flops (just like the right-looking algorithm).

▾ Solution

Consider the iteration where A_{TL} is (initially) $k \times k$. Then

- Solving $L_{00}u_{01} = a_{21}$ requires approximately k^2 flops.
- Updating $\alpha_{11} := \alpha_{11} - a_{10}^T a_{01}$ requires approximately $2k$ flops, which we will ignore.
- Updating $a_{21} := a_{21} - A_{20}a_{01}$ requires approximately $2(m-k-1)k$ flops.
- Updating $a_{21} := a_{21}/\alpha_{11}$ requires approximately $(m-k-1)$ flops, which we will ignore.

Thus, the total cost is given by, approximately,

$$\sum_{k=0}^{n-1} \left(k^2 + 2(m-k-1)k\right) \text{ flops.}$$

Let us now simplify this:

$$\begin{array}{l} \sum_{k=0}^{n-1} \left(k^2 + 2(m-k-1)k\right) \\ \quad = \quad < \text{ algebra } > \\ \sum_{k=0}^{n-1} k^2 + 2\sum_{k=0}^{n-1}(m-k-1)k \\ \quad = \quad < \text{ algebra } > \\ \sum_{k=0}^{n-1} 2(m-1)k - \sum_{k=0}^{n-1} k^2 \\ \quad \approx \quad < \sum_{j=0}^{n-1} j \approx n^2/2 \text{ and } \sum_{j=0}^{n-1} j^2 \approx n^3/3 > \\ (m-1)n^2 - \frac{1}{3}n^3 \end{array}$$

Had we not ignored the cost of $\alpha_{11} := \alpha_{11} - a_{10}^T a_{01}$, which approximately $2k$, then the result would have been approximately

$$mn^2 - \frac{1}{3}n^3$$

instead of $(m-1)n^2 - \frac{1}{3}n^3$, which is identical to that of the right-looking algorithm in Figure 5.2.2.1. This makes sense, since the two algorithms perform the same operations in a different order.

Of course, regardless,

$$(m-1)n^2 - \frac{1}{3}n^3 \approx mn^2 - \frac{1}{3}n^3$$

if m is large.

Remark 5.2.3.6. A careful analysis would show that the left- and right-looking algorithms perform the exact same operations with the same elements of A, except in a different order. Thus, it is no surprise that the costs of these algorithms are the same.

Ponder This 5.2.3.3. If A is $m \times m$ (square!), then yet another algorithm can be derived by partitioning A, L, and U so that

$$A = \begin{pmatrix} A_{00} & a_{01} \\ a_{10}^T & \alpha_{11} \end{pmatrix}, L = \begin{pmatrix} L_{00} & 0 \\ l_{10}^T & 1 \end{pmatrix}, U = \begin{pmatrix} U_{00} & u_{01} \\ 0 & v_{11} \end{pmatrix}.$$

Assume that L_{00} and U_{00} have already been computed in previous iterations, and determine how to compute u_{01}, l_{10}^T, and v_{11} in the current iteration. Then fill in the algorithm:

$A =$ LU-bordered(A)

$A \rightarrow \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right)$

A_{TL} is 0×0

while $n(A_{TL}) < n(A)$

$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right) \rightarrow \left(\begin{array}{cc|c} A_{00} & a_{01} & A_{02} \\ a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array}\right)$

$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right) \leftarrow \left(\begin{array}{cc|c} A_{00} & a_{01} & A_{02} \\ a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array}\right)$

endwhile

This algorithm is often called the *bordered* LU factorization algorithm.

Next, modify the proof of Theorem 5.2.3.4 to show the existence of the LU factorization *when A is square* and has nonsingular leading principal submatrices.

Finally, show that this bordered algorithm also requires approximately $2m^3/3$ flops.

Homework 5.2.3.4. Implement the algorithm given in Figure 5.2.3.5 as

```
function [ A_out ] = LU_left_looking( A )
```

by completing the code in `Assignments/Week05/matlab/LU_left_looking.m`. Input is an $m \times n$ matrix A. Output is the matrix A that has been overwritten by the LU factorization. You may want to use `Assignments/Week05/matlab/test_LU_left_looking.m` to check your implementation.

▸ Solution